Data and Artificial Intelligence
Cyber Shujaa Program

Week 12 Assignment
Generative AI and RAG

Student Name: Rodney Roy Gitonga
Student ID: CS-DA03-26025

Contents

Contents 1
Introduction 1
Tasks Completed 2
Step 1: Data Loading & Exploration 2
Step 2: Model Training (Embeddings & Vector Store) 2
Step 3: Evaluation Results & Querying 3
Link to Code: 4
Conclusion 4

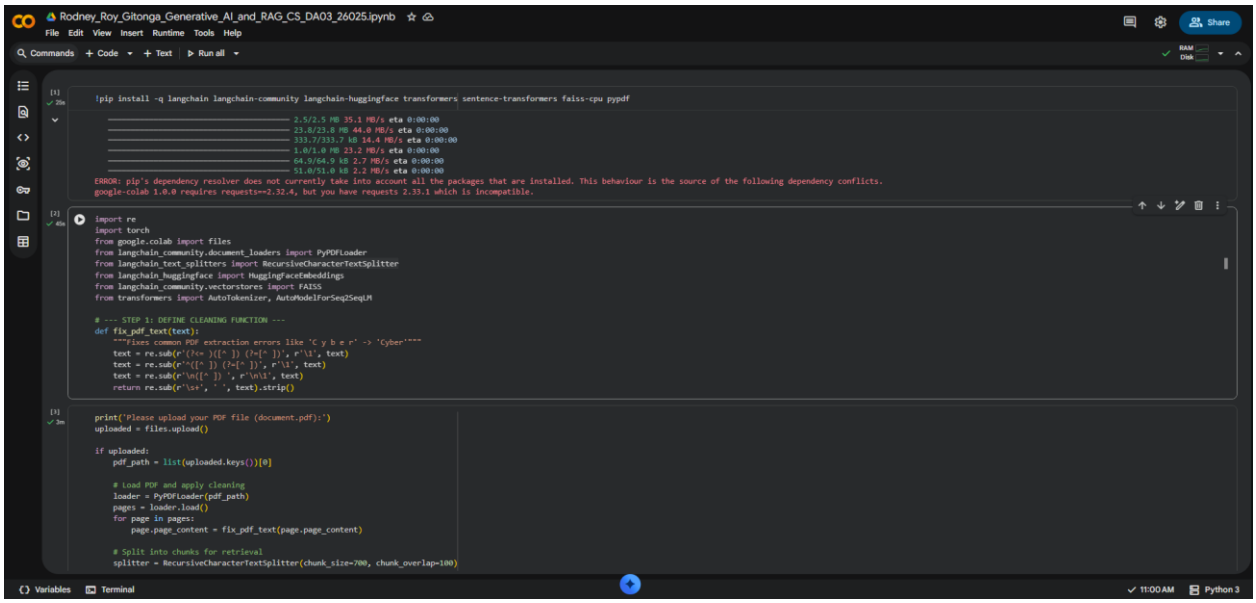
Introduction

This project demonstrates the practical application of Generative AI through the implementation of a Retrieval-Augmented Generation (RAG) pipeline. While Large Language Models (LLMs) are powerful, they are prone to hallucinations and lack access to private or highly specific, real-time data. RAG solves this by combining information retrieval with text generation. In this assignment, I built a pipeline that extracts text from a PDF, splits it into manageable chunks, converts those chunks into mathematical vectors (embeddings) using Sentence-Transformers, and stores them in a FAISS vector database. When a user asks a question, the system retrieves the most relevant document chunks and feeds them as context to a generative LLM (FLAN-T5) to produce an accurate, highly-grounded answer.

Tasks Completed

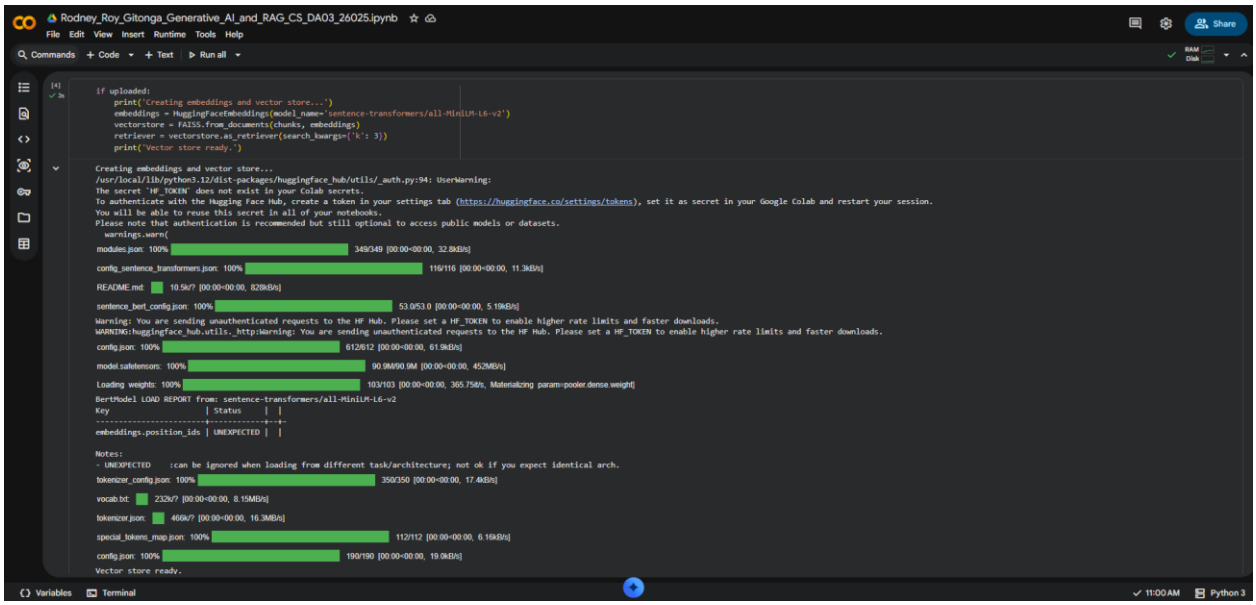
Step 1: Data Loading & Exploration

Raw PDF extraction often results in poorly formatted text (e.g., strange spacing). I used a regex-based cleaning function to standardize the text. Then, using RecursiveCharacterTextSplitter, I split the document into chunks of 700 characters with an overlap of 100 characters. Chunking is essential because LLMs have strict token limits; we cannot pass an entire book or long document into the model at once. The overlap ensures that sentences or concepts aren't cut off abruptly, preserving the semantic context between chunks.



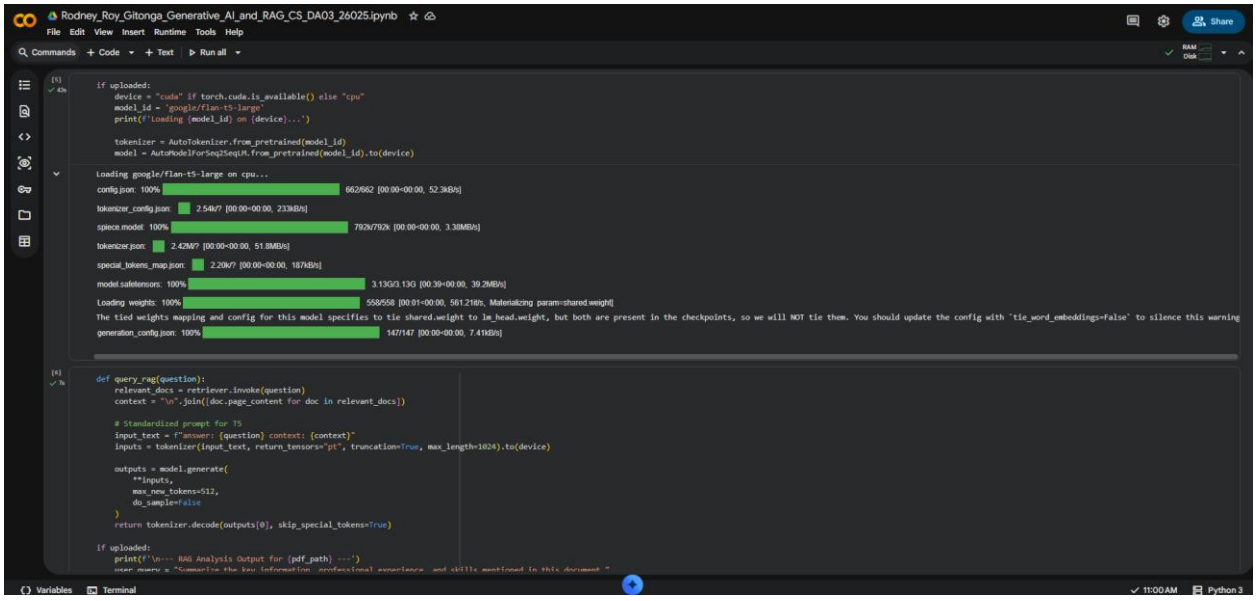
Step 2: Model Training (Embeddings & Vector Store)

I used the all-MiniLM-L6-v2 model from Hugging Face to generate embeddings. Embeddings are dense numerical vectors that capture the semantic meaning of the text. I then stored these embeddings in FAISS (Facebook AI Similarity Search), an efficient vector database optimized for rapid similarity search. When a query is made, FAISS compares the query's embedding against all document chunk embeddings to find the top k (in this case, 3) most relevant pieces of information.



Step 3: Evaluation Results & Querying

I manually loaded the google/flan-t5-large generative model. The RAG query function takes the user's question, searches the FAISS database for the top 3 relevant chunks, and concatenates them into a context variable. Using prompt engineering, I structured the input as "answer based on context: [question] context: [retrieved chunks]". This strict prompt structure acts as a guardrail, forcing the FLAN-T5 model to synthesize its answer strictly from the retrieved document data rather than relying on its pre-trained, generic knowledge, significantly reducing the chance of hallucination.



```python
def query_rag(question):
    relevant_docs = retriever.invoke(question)
    context = "\n".join([doc.page_content for doc in relevant_docs])

    # Standardized prompt for T5
    input_text = f"answer: {question} context: {context}"
    inputs = tokenizer(input_text, return_tensors="pt", truncation=True, max_length=1024).to(device)

    outputs = model.generate(
        **inputs,
        max_new_tokens=512,
        do_sample=False
    )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

if uploaded:
    print(f'\n--- RAG Analysis Output for {pdf_path} ---')
    user_query = "Summarize the key information, professional experience, and skills mentioned in this document."
    response = query_rag(user_query)
    print(response)
```

```
--- RAG Analysis Output for 01_Python_Introduction.pdf ---
Baraa Khatib Salkini is a Python programmer.
```

Terminal ✓ 11:00 AM Python 3

Link to Code:

https://colab.research.google.com/drive/1kkqa-sJsD5uLR1iZrdkoWQQkw2I_Slp2?usp=sharing

Conclusion

Through this assignment, I learned how to successfully bridge the gap between static documents and generative AI by building a functional RAG pipeline. I discovered that the quality of an LLM's output is highly dependent on the quality of the retrieval system (chunking strategy, embedding model, and vector search). Furthermore, I gained hands-on experience with LangChain's orchestration capabilities and saw firsthand how prompt engineering—specifically instructing the model to rely only on the provided context—forces an otherwise generic LLM to provide highly specific, document-grounded answers.